

EduMate - Guide de déploiement Docker et Application Mobile

Introduction

Ce guide couvre le déploiement complet de la plateforme EduMate en production :

- Les services backend (microservices)
- Les bases de données (PostgreSQL, MongoDB, Qdrant, Ganache)
- L'application web frontend
- L'application mobile Android (APK)

Destiné à : Étudiants de 2ème année - Équipes de déploiement et maintenance

Prérequis :

- Docker Desktop 20.10+ installé
- Docker Compose 2.0+ installé
- 8 Go de RAM minimum
- 20 Go d'espace disque libre
- Node.js 20+ (pour build sans Docker)
- Docker CLI (en ligne de commande)

Architecture Docker - Vue d'ensemble

Services déployés (10 conteneurs)

Le fichier `docker-compose.yml` orchestre 10 services interconnectés :

Service	Image/Dockerfile	Port exposé	Rôle
postgres	postgres:16-alpine	5432	Base de données relationnelle principale
mongodb	mongo:7	27017	Base NoSQL pour messages et logs
qdrant	qdrant/qdrant:latest	6333	Base vectorielle pour recherche sémantique
ganache	trufflesuite/ganache:latest	8545	Blockchain Ethereum locale
auth-service	./services/auth-service	3001	Authentification JWT et profils
message-service	./services/message-service	3002	Chat temps réel WebSocket
blockchain-service	./services/blockchain-service	3003	Interface smart contracts
cv-parser-service	./services/cv-parser-service	5001	Parsing CV avec IA
chatbot-service	./services/chatbot-service	4000	Agent conversationnel
web	./apps/web	5173	Frontend React

Réseau Docker

Tous les services communiquent via le réseau interne `edumate-network` (bridge). Les services backend accèdent aux bases de données par leurs noms de service (ex: `postgres:5432` , `mongodb:27017`).

Fichier docker-compose.yml - Explication détaillée

Structure générale

```
version: '3.8'
networks:
  edumate-network:
    driver: bridge
volumes:
  postgres-data:
  mongodb-data:
  qdrant-data:
```

Réseaux : Un seul réseau bridge `edumate-network` pour la communication inter-services.

Volumes : Volumes nommés pour persister les données des bases de données entre redémarrages.

Service PostgreSQL

```
postgres:
  image: postgres:16-alpine
  container_name: edumate-postgres
  environment:
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB: ${POSTGRES_DB}
  ports:
    - "5432:5432"
  volumes:
    - postgres-data:/var/lib/postgresql/data
  networks:
    - edumate-network
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Détails :

- Image officielle Alpine (légère)
- Variables d'environnement injectées depuis `.env`
- Volume persistant pour conserver les données
- Healthcheck pour vérifier que PostgreSQL est prêt avant de démarrer les services dépendants

Service MongoDB

```
mongodb:
  image: mongo:7
  container_name: edumate-mongodb
  environment:
    MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
    MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
  ports:
    - "27017:27017"
  volumes:
    - mongodb-data:/data/db
  networks:
    - edumate-network
```

Détails :

- Image officielle MongoDB 7
- Authentification activée avec user/password
- Données stockées dans `/data/db` (volume persistant)

Service Qdrant (Base vectorielle)

```
qdrant:
  image: qdrant/qdrant:latest
  container_name: edumate-qdrant
  ports:
    - "6333:6333"
    - "6334:6334"
  volumes:
    - qdrant-data:/qdrant/storage
  networks:
    - edumate-network
```

Détails :

- Port 6333 : API REST
- Port 6334 : gRPC (optionnel)
- Stockage vectoriel pour embeddings IA

Service Ganache (Blockchain)

```
ganache:
  image: trufflesuite/ganache:latest
  container_name: edumate-ganache
  command: [
    "--mnemonic", "myth like bonus scare over problem client lizard pioneer submit female collect",
    "--accounts", "10",
    "--defaultBalanceEther", "1000",
```

```
    "--gasLimit", "10000000",
    "--hardfork", "london"
]
ports:
  - "8545:8545"
networks:
  - edumate-network
```

Détails :

- Blockchain Ethereum locale pour développement
- 10 comptes précréés avec 1000 ETH chacun
- Mnémonique fixe pour reproductibilité
- Hardfork London (support EIP-1559)

Service auth-service (Backend Node.js)

```
auth-service:
  build:
    context: ./services/auth-service
    dockerfile: Dockerfile
  container_name: edumate-auth
  environment:
    NODE_ENV: production
    PORT: 3001
    DATABASE_URL:
      postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
    JWT_SECRET: ${JWT_SECRET}
    OPENROUTER_API_KEY: ${OPENROUTER_API_KEY}
  ports:
    - "3001:3001"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - edumate-network
  restart: unless-stopped
```

Détails :

- Build depuis Dockerfile local
- Attend que PostgreSQL soit "healthy" avant de démarrer
- Connexion BDD via URL interne Docker (postgres:5432)
- Redémarrage automatique en cas de crash

Service blockchain-service (Backend Python)

```
blockchain-service:
  build:
    context: ./services/blockchain-service
    dockerfile: Dockerfile
```

```
container_name: edumate-blockchain
environment:
  WEB3_PROVIDER_URL: http://ganache:8545
  PRIVATE_KEY: ${PRIVATE_KEY}
ports:
  - "3003:3003"
depends_on:
  - ganache
networks:
  - edumate-network
```

Détails :

- Service Python avec Web3.py
- Connexion à Ganache via réseau interne
- Clé privée pour déploiement de smart contracts

Service web (Frontend React)

```
web:
  build:
    context: ./apps/web
    dockerfile: Dockerfile
  container_name: edumate-web
  environment:
    VITE_API_URL: http://localhost:3001
    VITE_BLOCKCHAIN_URL: http://localhost:3003
  ports:
    - "5173:5173"
  depends_on:
    - auth-service
    - blockchain-service
  networks:
    - edumate-network
```

Détails :

- Build production avec Vite
- Variables d'environnement pour URLs API
- Serveur Nginx ou serveur dev Vite selon Dockerfile

Dockerfiles - Analyse détaillée

Dockerfile auth-service (Backend Node.js)

```
# services/auth-service/Dockerfile
FROM node:20-alpine

WORKDIR /app

# Installation des dépendances système
```

```

RUN apk add --no-cache python3 make g++

# Copie des fichiers de dépendances
COPY package*.json ./

# Installation des dépendances npm
RUN npm ci --only=production

# Copie du code source
COPY . .

# Exposition du port
EXPOSE 3001

# Commande de démarrage
CMD ["node", "src/index.js"]

```

Explications :

- Image Alpine (légère, ~50 Mo au lieu de 300 Mo)
- `npm ci` au lieu de `npm install` (installation déterministe)
- Mode `--only=production` (exclut devDependencies)
- Multi-stage possible pour optimiser la taille

Version multi-stage optimisée :

```

# Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # Si build nécessaire

# Production stage
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY package*.json ./
EXPOSE 3001
CMD ["node", "dist/index.js"]

```

Dockerfile blockchain-service (Backend Python)

```

# services/blockchain-service/Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Installation des dépendances système

```

```

RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Copie des requirements
COPY requirements.txt .

# Installation des dépendances Python
RUN pip install --no-cache-dir -r requirements.txt

# Copie du code source
COPY . .

# Exposition du port
EXPOSE 3003

# Commande de démarrage
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "3003"]

```

Explications :

- Image Python slim (réduit la taille)
- Installation des dépendances système si nécessaire (gcc pour compilations)
- `pip install --no-cache-dir` pour réduire la taille de l'image
- Uvicorn pour serveur ASGI (FastAPI)

Dockerfile cv-parser-service (Python + IA)

```

# services/cv-parser-service/Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Installation de poppler-utils pour PDF parsing
RUN apt-get update && apt-get install -y \
    poppler-utils \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5001

CMD ["python", "src/app.py"]

```

Explications :

- `poppler-utils` nécessaire pour extraire texte des PDF
- Dépendances IA (mistralai, pdfplumber)

Dockerfile web (Frontend React)

Version développement :

```
# apps/web/Dockerfile
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Version production (recommandée) :

```
# Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Configuration Nginx (apps/web/nginx.conf):

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://edumate-auth:3001;
        proxy_set_header Host $host;
    }
}
```



```
    proxy_set_header X-Real-IP $remote_addr;
}
}
```

Déploiement en production - Étapes détaillées

1. Préparation de l'environnement

```
# Cloner le dépôt
git clone https://github.com/NEXUS-AI-Innovation-lab/EduMate.git
cd EduMate

# Créer le fichier .env
cp .env.docker.example .env
nano .env # Éditer avec vos valeurs
```

Fichier .env requis :

```
# PostgreSQL
POSTGRES_USER=edumate_user
POSTGRES_PASSWORD=edumate_password
POSTGRES_DB=edumate
POSTGRES_PORT=5432

# MongoDB
MONGO_USER=edumate_user
MONGO_PASSWORD=edumate_password
MONGO_DB=edumate
MONGO_PORT=27017

# JWT - Remplace par une clé sécurisée
JWT_SECRET=your-super-secret-jwt-key-change-this-in-production
JWT_EXPIRE=7d

# Blockchain
GANACHE_PORT=8545
GANACHE_MNEMONIC=test test test test test test test test test test test junk
BLOCKCHAIN_PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb476caded64602d3f5ee1e8c1c67

# IA Services - À remplir avec tes vraies clés API
MISTRAL_API_KEY=$MISTRAL_API_KEY
OPENROUTER_API_KEY=$OPENROUTER_API_KEY
```

2. Build et lancement des conteneurs

```
# Build toutes les images
docker-compose build

# Lancement en mode détaché
```

```

docker-compose up -d

# Vérifier les logs
docker-compose logs -f

# Vérifier l'état des services
docker-compose ps

```

Sortie attendue :

NAME	COMMAND	SERVICE	STATUS	PORTS
edumate-auth	"docker-entrypoint.s..."	auth-service	running	
0.0.0.0:3001->3001/tcp				
edumate-blockchain	"uvicorn src.main:ap..."	blockchain-service	running	
0.0.0.0:3003->3003/tcp				
edumate-ganache	"node /app/ganache-c..."	ganache	running	
0.0.0.0:8545->8545/tcp				
edumate-mongodb	"docker-entrypoint.s..."	mongodb	running	
0.0.0.0:27017->27017/tcp				
edumate-postgres	"docker-entrypoint.s..."	postgres	running	
0.0.0.0:5432->5432/tcp				
edumate-qdrant	"/qdrant --config-pa..."	qdrant	running	
0.0.0.0:6333-6334->6333-6334/tcp				
edumate-web	"docker-entrypoint.s..."	web	running	
0.0.0.0:5173->5173/tcp				

3. Initialisation des bases de données

```

# Créer les tables PostgreSQL (migrations)
docker exec -it edumate-auth npm run migrate

# Vérifier PostgreSQL
docker exec -it edumate-postgres psql -U edumate_user -d edumate -c "\dt"

# Vérifier MongoDB
docker exec -it edumate-mongodb mongosh -u edumate_user -p edumate_password --
authenticationDatabase admin

```

4. Déploiement des smart contracts

```

# Déployer EduToken et BookingEscrow sur Ganache
docker exec -it edumate-blockchain python scripts/deploy_contracts.py

# Vérifier le déploiement
curl http://localhost:3003/health

```

5. Tests de santé des services

```
# Auth service
curl http://localhost:3001/health
# Réponse attendue: {"status":"ok","database":"connected"}

# Blockchain service
curl http://localhost:3003/health
# Réponse attendue: {"status":"ok","ganache":"connected","contracts":"deployed"}

# Frontend
curl http://localhost:5173
# Réponse attendue: HTML de la page d'accueil
```

Optimisation et Production-Ready

1. Health Checks avancés

Ajouter dans `docker-compose.yml` :

```
auth-service:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:3001/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s
```

2. Logs centralisés

```
services:
  auth-service:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

3. Limites de ressources

```
auth-service:
  deploy:
    resources:
      limits:
        cpus: '1'
        memory: 512M
      reservations:
        cpus: '0.5'
        memory: 256M
```

4. Sauvegarde automatique PostgreSQL

```
# Script backup.sh
#!/bin/bash
docker exec edumate-postgres pg_dump -U edumate_user edumate > backup-$(date +%Y%m%d).sql
```

Cron job (Linux) :

```
0 2 * * * /home/user/edumate/backup.sh
```

5. Monitoring avec Portainer

```
portainer:
  image: portainer/portainer-ce:latest
  ports:
    - "9000:9000"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - portainer-data:/data
```

Commandes utiles pour maintenance

```
# Arrêter tous les services
docker-compose down

# Arrêter et supprimer volumes (ATTENTION: perte données)
docker-compose down -v

# Rebuild un seul service
docker-compose up -d --build auth-service

# Voir logs temps réel
docker-compose logs -f auth-service

# Redémarrer service spécifique
docker-compose restart auth-service

# Accéder au shell d'un conteneur
docker exec -it edumate-auth sh

# Statistiques ressources
docker stats

# Nettoyer images inutilisées
docker system prune -a
```

Troubleshooting commun

Erreur: Port déjà utilisé

```
# Windows: trouver processus
netstat -ano | findstr :3001
taskkill /PID <PID> /F

# Linux
lsof -ti:3001 | xargs kill -9
```

Erreur: Cannot connect to database

```
# Vérifier que PostgreSQL est démarré
docker-compose ps postgres

# Vérifier les logs
docker-compose logs postgres

# Tester connexion
docker exec -it edumate-postgres psql -U edumate_user -d edumate
```

Erreur: Blockchain service timeout

```
# Vérifier Ganache
docker-compose logs ganache

# Redéployer contracts
docker exec -it edumate-blockchain python scripts/deploy_contracts.py
```

Auteurs: Équipe EduMate UPEC (Hasmir BOINALI & Thanh Uyen NGUYEN)